Individual Report

AWS Infrastructure and Security Reflection

Course: Cloudutveckling (MU25)
Student: Vitaliy Beletskiy
Date: 2026-05-22

Introduction

In this project, two different AWS infrastructures for a full-stack web application were implemented and tested. The goal of the project was to study modern cloud solutions, networking, security, and different ways to implement backend and frontend in AWS.

The application consists of a frontend part, backend API, and database. The frontend is hosted in Amazon S3 and accessible through CloudFront. The backend was written in Spring Boot and deployed in Docker through ECS/EC2. There is also a serverless solution using AWS Lambda and API Gateway. MySQL database in Amazon RDS is used for data storage.

During the project, the following topics were studied:

- working with VPC and Security Groups;
- configuring IAM roles and permissions;
- interaction between AWS services;
- differences between traditional container architecture and serverless architecture;
- security and public access restrictions;
- file upload using S3 and pre-signed URLs.

The main idea of the project was to build a working application and understand how different AWS services interact with each other and how to organize secure cloud infrastructure correctly.

Frontend Infrastructure (S3 + CloudFront)

The frontend part of the application is hosted in S3 as a static website. CloudFront is used for content delivery and HTTPS support.

Direct access to the S3 bucket is closed. Instead, Origin Access Control (OAC) is used, allowing only CloudFront to access bucket content.

The frontend communicates with the backend through API Gateway by sending HTTP requests to API endpoints.

This approach turned out to be simple, inexpensive, and convenient for deployment. It also scales well and does not require a separate web server for the frontend part of the application.

Infrastructure 1 – Container-based Architecture (ECS/EC2)

The first infrastructure was built using a container-based approach. The backend application is running in Docker containers on an EC2 instance managed by ECS.

The Spring Boot application provides a REST API and communicates with a MySQL database in RDS. Lambda in API Gateway is used to access the backend and forward HTTP requests to backend endpoints.

Network access between resources was controlled using Security Groups.

The main advantage of this architecture is a high level of control over the system. Containers simplify deployment and allow moving the application between environments. This approach is also close to real production environments.

However, ECS infrastructure configuration turned out to be quite complex, especially deployment management. During the project, there were also problems with ECS agent and container connectivity, which showed that container-based architecture requires a deeper understanding of AWS infrastructure.

Infrastructure 2 – Serverless Architecture (Lambda)

The second infrastructure was built using a serverless approach based on Lambda and API Gateway.

In this case, backend logic runs inside Lambda functions without the need to manage servers or containers. API Gateway is used to process HTTP requests and invoke the corresponding Lambda functions.

The serverless architecture was used for different tasks, including creation of simple stateless endpoints returning information such as client IP address or hardcoded JSON responses and generation of pre-signed URLs for uploading/listing/downloading images. Amazon S3 is used for image storage.

Serverless services provide simple deployment, work well for small independent functions, and help reduce infrastructure cost because resources are used only during request execution.

However, during the project it became clear that when the number of Lambda functions increases, the architecture becomes more fragmented. Debugging and monitoring in serverless architecture can also become more difficult when the number of Lambda functions grows.

Security and Access

IAM roles, Security Groups, and separation of public/private access are used to protect AWS resources.

IAM roles are used to provide permissions to AWS services. For example, Lambda functions accessed S3 through IAM roles. Permissions are given only for AWS resources and actions that are really needed for the functions.

Security Groups are used as virtual firewalls to control network traffic between resources. For example, the backend application on EC2 is accessible only for Lambda functions, and access to the RDS database is allowed only for the backend application.

Some resources are public, such as CloudFront and API Gateway, because they must be accessible to users through the internet. Backend services and the database are placed inside VPC and have limited access.

RDS does not have a public IP and is accessible only from allowed resources inside the infrastructure. The S3 bucket is also not directly public because access to frontend files is performed through CloudFront using Origin Access Control (OAC).

During the project, it became clear that correct access control configuration is one of the most important parts of cloud infrastructure. Even a small mistake in IAM permissions or Security Groups can lead to connectivity or security problems.

Reflection

During the project, practical experience with AWS infrastructure, networking, and deployment was gained. Comparing container-based architecture and serverless architecture was especially useful because both approaches have different advantages and disadvantages.

Container-based architecture with ECS/EC2 provides more control over system configuration and is closer to traditional backend infrastructure. However, configuration and troubleshooting of such systems turned out to be quite complex, especially when working with deployment management and network configuration.

Serverless architecture with Lambda turned out to be simpler for small independent services. Deployment is easier, and AWS automatically manages Lambda function execution depending on the number of requests. At the same time, debugging and monitoring of serverless applications can become more difficult when the number of Lambda functions increases.

The project also helped to better understand IAM roles, Security Groups, VPC, and interaction between AWS services. In practice, it became clear how important correct access permissions and network security configuration are.

Overall, the project provided good practical experience in building cloud infrastructure and showed differences between several deployment approaches in AWS.